

CMP513 - Computer Graphics and Image Processing

Tutorial Activity

Objective: Compare mathematical precision, edge-case safety, and computational efficiency of basic vs. optimized DDA variants.

Variant 1 - Basic Rounded DDA

Loop Bounds: $i = 0$ to $i < \text{steps}$ (Exactly steps iterations)

```
1. Read the input of two end points (X1,Y1) and (X2,Y2)
2. Calculate the difference between two end points
   dx = X2 - X1
   dy = Y2 - Y1
3. Identify the number of steps based on the calculation in step 2
   if dx > dy,
       then more steps in x coordinate;
   else
       More steps in y coordinate.

   if (absolute(dx) > absolute(dy))
       steps = absolute(dx);
   else
       steps = absolute(dy);
4. Calculate the increment in x coordinate and y coordinate.
   Xincrement = dx / (float) steps;
   Yincrement = dy / (float) steps;
5. Initialize x=x1 and y=y1
6. Put the pixel by incrementing x and y coordinates accordingly
   for (i=0; i < Steps; i++)
   {
       putpixel(Round(x), Round(y));
       x = x + Xincrement;  y = y + Yincrement;
   }
7. Stop
```

Variant 2 - Optimized Sign-Bias DDA

Loop bounds: step = 0 to step <= length (Exactly length + 1 iterations)

```
1. Read end points (x1, y1) and (x2, y2).
   If x1 = x2 and y1 = y2.
       Plot Point.
       Exit.
2.  $\Delta x = |x_2 - x_1|$  and  $\Delta y = |y_2 - y_1|$ 
3. If ( $\Delta x \geq \Delta y$ )
       length =  $\Delta x$  else
       length =  $\Delta y$ 
4.  $\Delta x = (x_2 - x_1) / \text{length}$  ,
    $\Delta y = (y_2 - y_1) / \text{length}$ 
5. Calculate
    $x = x_1 + 0.5 * \text{Sign}(\Delta x)$  and
    $y = y_1 + 0.5 * \text{Sign}(\Delta y)$ 
6. Loop for step=0, step  $\leq$  length, step =step+1
   {
       plot (Integer (x), Integer (y))
        $x = x + \Delta x$ ,  $y = y + \Delta y$ 
   }
7. Stop
```

Perform the calculation for following coordinates and plot the graphs.

Line 1 - Start (0,0) to End (9,2) (Positive slope)

Line 2 - Start (5,8) to End (2,0) (Negative slope)

Format of calculation table

For variant 1

iteration	Current x	Current y	Evaluated Round(x), Round(y)	Pixel Plotted	Next x	Next y
i = 0	0.0	0.0	Round(0.0), Round(0.0)	(0, 0)	1.0	0.222
i = 1	1.0	0.222	Round(1.0), Round(0.222)	(1, 0)	2.0	0.444

For variant 2

iteration	Current x	Current y	Evaluated Integer(x), Integer(y)	Pixel Plotted	Next x	Next y
step = 0	0.0	0.0	Integer(0.0), Integer(0.0)	(0, 0)	1.0	0.222
step = 1	1.0	0.222	Integer(1.0), Integer(0.22)	(1, 0)	2.0	0.444

Attempt the following points based on your observations from above calculations

1. Fill in the iteration values up to when the loop finishes. Identify the exact coordinates of the final pixel plotted.
2. Notice how the termination condition cuts the execution off before the last set of coordinates can be processed.
3. Note: Since $|dy| > |dx|$, your steps will be guided by y. Pay close attention to how $\text{Sign}(\Delta x)$ and $\text{Sign}(\Delta y)$ evaluate to negative numbers altering your 0.5 pre-bias.

Descriptive analytical points to address

1. For the point line consider the input as Start (4,4) to End (4,4). Write out the variable assignment sequence for both algorithms. Detail which line of code causes Variant 1 to crash and how Variant 2 safely addresses it.
2. Imagine a rendering loop where small, decimal variations are passed to a line drawing function (e.g., animating a spinning wheel). Deduce which of the two variants will maintain stable line thickness without experiencing blinking pixels or structural warping during movement. Focus your argument on the stability of the integer truncation bias.
3. From a microarchitecture standpoint, why is executing $\text{Integer}(x)$ (bit truncation) significantly faster than calling a standard math library $\text{Round}(x)$ function?
4. Compare variant 1 and variant 2 based on features / characteristics that you have observed.